Issue Tracker System: Business Requirements Document

Author: Dinu Robin | Version 1.0 | 28 October 2026

1. Purpose

This document describes the business requirements, scope, design and deployment of the Issue Tracker System. It is written so that another developer can understand what the application does, why it works the way it does, and how to run and deploy it without help from the author.

2. Background and Objectives

Teams need one place to report problems, decide who owns them, track their progress and discuss them. Without it, issues get lost in chat threads and email.

The objectives of the system are:

- Let people report issues and see all open work in one place.
- Let an administrator control who owns each issue and what state it is in.
- Keep a record of the discussion on every issue.
- Give the administrator a quick view of overall progress through status counts.
- Keep the stack simple and free to host.

3. Scope

In scope

- User registration and login
- A separate administrator login
- Creating, editing and deleting issues
- Assigning issues to users (admin only)
- Status tracking: Open, In Progress, Closed (admin only)
- Comments on issues
- An administrator dashboard with status counts

- Deployment to a public URL

Out of scope

- Email or push notifications
- File attachments on issues
- Password reset and social login
- Multiple projects or teams
- Audit history of changes

4. Users and Roles

Role	Description
User	Registers through the app. Reports issues, edits and deletes their own, browses all issues and comments on any of them.
Admin	Created by the system owner directly in the database, never through the app. Manages every issue and comment and sees the status counts.

Users and admins have separate login pages and separate dashboards.

5. Functional Requirements

ID	Requirement
FR-1	A visitor can register with full name, email and password.
FR-2	After registering, the user is sent to the user login page.
FR-3	A user logs in at the user login page. An admin account is rejected there.
FR-4	An admin logs in at a separate admin login page. A non-admin account is rejected there.

ID	Requirement
FR-5	A logged-in user can create an issue with a title and a description. New issues start as Open and unassigned.
FR-6	A user can edit the title and description of their own issues.
FR-7	A user can delete their own issues. An admin can delete any issue.
FR-8	A user can view all issues and switch between All Issues and My Issues.
FR-9	An admin can assign any issue to any registered user, or unassign it.
FR-10	An admin can set an issue's status to Open, In Progress or Closed.
FR-11	Any logged-in user can comment on any issue.
FR-12	A user can delete their own comments. An admin can delete any comment.
FR-13	The admin dashboard shows counts for total, open, in progress and closed issues, and can filter the issue list by status.
FR-14	The user dashboard does not show the status counts.
FR-15	Each role is redirected away from the other role's dashboard.
FR-16	Closing the browser tab or window ends the session.

6. Business Rules

- Only an admin can change the status or the assignee of an issue.
- A user can only edit or delete issues they created.

- Nobody can change their own role through the application. Admins are promoted manually by the system owner.
- Deleting an issue also deletes its comments.
- Every signup creates a profile with the role User.

These rules are enforced in the database with Row Level Security policies and a trigger. The interface hides controls to match, but it is not the thing that protects the data.

7. Non-Functional Requirements

Area	Requirement
Security	Passwords are stored hashed by Supabase Auth. Access is controlled by RLS. Only the public anon key is used in the browser. No privileged key is shipped to the client.
Usability	Responsive layout for desktop and mobile. Clear error messages on failed actions.
Performance	Dashboard data loads in a single round of requests. Suitable for small teams and demo-sized data.
Availability	Depends on the free tiers of Vercel and Supabase. A free Supabase project pauses after about a week of inactivity and must be restored from its dashboard.
Maintainability	Small isolated components, shared hooks and styles, and a single schema file.
Cost	Zero. Every service used has a free plan that is sufficient.

8. System Design

Architecture

The application is a single-page React app built with Vite. It has no custom server. The browser talks directly to Supabase using the Supabase JavaScript client. Supabase provides authentication, the Postgres database and an automatically generated REST API.

```
Browser (React SPA on Vercel)
   |
   |  supabase-js
   v
Supabase
   |-- Auth       email and password, JWT sessions
   |-- Postgres   profiles, issues, comments, RLS
   `-- REST API   generated from the tables
```

Technology

Layer	Choice
Frontend	React with Vite and React Router
Backend	Supabase (Postgres, Auth, REST API)
Authorization	Row Level Security with a role column on profiles
Hosting	Vercel for the frontend, Supabase for data and auth
Source control	GitHub

A separate backend such as Django was not used because Vercel and GitHub Pages only host frontends. A custom backend would need a third host and extra configuration, while Supabase covers every requirement with no server to deploy.

Data Model

Table	Key columns
profiles	id (same as the auth user id), email, full_name, role (user or admin), created_at
issues	id, title, description, status (open, in_progress, closed), created_by, assignee_id, created_at, updated_at
comments	id, issue_id, author_id, body, created_at

Relationships: issues.created_by and issues.assignee_id point to profiles. comments.issue_id points to issues and cascades on delete. comments.author_id points to profiles. A database trigger creates the profile row whenever a user signs up.

Permission Matrix

Action	User	Admin
View all issues	Yes	Yes
Create an issue	Yes	Yes
Edit title and description	Own issues	Not offered in the interface
Delete an issue	Own issues	Any issue
Assign an issue	No	Yes
Change status	No	Yes
Comment on an issue	Yes	Yes
Delete a comment	Own comments	Any comment
See dashboard counts	No	Yes

Screens

Route	Screen	Access
/login	User login	Public
/register	Registration	Public
/admin/login	Admin login	Public
/dashboard	User dashboard	Users
/admin	Admin dashboard with counts	Admins
/issues/:id	Issue details	Logged in

9. Deployment

Where it is hosted

- The frontend is hosted on Vercel at the live URL listed in the README.
- The database, authentication and API are hosted on Supabase on the free plan.
- The source code is on GitHub, and Vercel deploys from it.

Deployment approach

Deployment is continuous and Git-based. Vercel is connected to the GitHub repository. Every push to the main branch triggers a new production build and deploy. No server is managed by hand. The database lives separately on Supabase, so deploying the frontend never touches the data.

Required services and accounts

Service	Purpose
GitHub	Source code and deployment trigger
Vercel	Builds and serves the frontend
Supabase	Database, authentication and API

Configuration

Variable	Where it is set	Description
VITE_SUPABASE_URL	Local .env file and Vercel environment variables	Supabase project URL
VITE_SUPABASE_ANON_KEY	Local .env file and Vercel environment variables	Supabase public anon or publishable key

Only the public key is used. The service role key and the database password must never be placed in the frontend or in a VITE_ variable.

Steps to deploy from scratch

1. Create a Supabase project.
2. Open the SQL Editor and run supabase/schema.sql. This creates the tables, the signup trigger, the grants and the RLS policies.
3. In Authentication, Sign In / Providers, keep the Email provider enabled and turn Confirm email off, so registration works without email delivery.
4. Copy the Project URL and the anon key from Project Settings, API.
5. Push the repository to GitHub. The repository contains vercel.json, which rewrites all routes to index.html so refreshes on pages such as /dashboard work.
6. In Vercel, import the repository. Keep the Vite defaults: build command npm run build, output directory dist.
7. Add VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY as environment variables and deploy.
8. In Supabase, Authentication, URL Configuration, set the Site URL to the Vercel URL.
9. Create the first admin: add a user in Authentication, Users (with Auto Confirm User ticked), then run update profiles set role = 'admin' where email = 'the admin email'.
10. Verify the live site: register a user, log in on both login pages, create an issue, assign it, change its status and add a comment.

Steps to update the application

- Code changes: commit and push to main. Vercel rebuilds and redeploys automatically, usually within a minute or two.

- Database changes: run the new SQL in the Supabase SQL Editor and add the same statements to supabase/schema.sql so the repository stays the source of truth.
- Environment variable changes: edit them in Vercel, then trigger a redeploy, because Vite bakes them into the build.
- Rollback: in Vercel, open Deployments and promote a previous deployment.

Operational notes

- A free Supabase project pauses after about a week without activity. The fix is to click Restore in the Supabase dashboard.
- The built-in Supabase email sender is rate limited, which is why email confirmation is turned off.
- Sessions are stored in sessionStorage, so users are logged out when they close the tab.

10. Assumptions and Constraints

- The application is intended for small teams and demonstration use.
- Free-tier limits of Vercel and Supabase apply.
- Email confirmation is disabled by design.
- Admins are created only through the Supabase dashboard.
- A modern browser with JavaScript enabled is required.

11. Risks

Risk	Impact	Mitigation
Supabase free project pauses after inactivity	The app shows errors until restored	Restore from the dashboard. Documented in the README.
Misconfigured RLS exposes data	Unauthorized reads or writes	All policies live in schema.sql and are tested with both roles.
Public sign-up allows spam accounts	Clutter in the user list	Acceptable for a demo. A CAPTCHA or invite flow could be added.

Risk	Impact	Mitigation
Environment variables missing in Vercel	Blank screen on the live site	Variables are listed in the README and the deployment steps.

12. Acceptance Criteria

- A new user can register, is sent to the login page, and can log in to the user dashboard.
- A user cannot see assign or status controls, and cannot edit or delete another user's issue.
- A user can read and comment on any issue.
- The admin can log in only through the admin login, sees the four count cards, and can assign, change status, delete and comment.
- A non-admin who opens /admin is redirected to /dashboard.
- Closing the tab logs the user out.
- The live Vercel URL works end to end and the README and this document match the running system.

13. Possible Future Enhancements

- Email notifications on assignment and new comments
- Issue priority, labels and due dates
- Search and sorting
- File attachments
- Activity history for each issue
- Password reset flow